

Projektowanie Efektywnych Algorytmów

Problem Komiwożacza (ATSP)

Marcel Cieśliński

Nr albumu: 280871

25 Marzec 2026

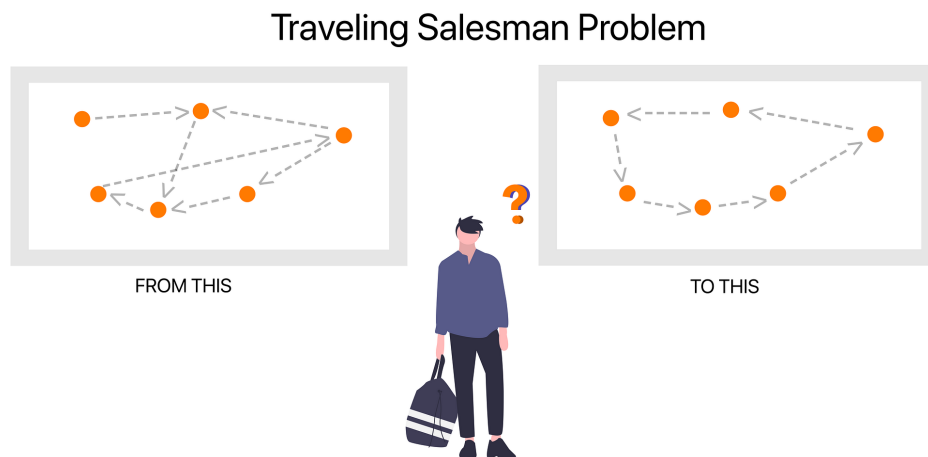
Kurs	Projektowanie Efektywnych Algorytmów - Projekt
Temat projektu	Problem Komiwożacza (ATSP) - Zadanie 1
Prowadzący	dr inż. Jarosław Mierzwa
Termin oddania	26 marca 2026 r.
Autor	Marcel Cieśliński

1 Wstęp

Celem projektu jest zbadanie problemu komiwojażera (ang. *Traveling Salesman Problem* – TSP) oraz znalezienie możliwie najkrótszej trasy dla zadanych danych. Głównym założeniem problemu jest to, że komiwojażer musi odwiedzić wszystkie N miast (wierzchołków) dokładnie jeden raz, a na koniec wrócić do punktu startowego. Koszt przejścia między poszczególnymi miastami opisuje macierz kosztów o rozmiarze $N \times N$.

Należy zwrócić uwagę na asymetryczny problem komiwojażera (ATSP), który różni się od klasycznego TSP tym, że koszt przejazdu między miastami może być różny w zależności od kierunku. Wariancja klasyczna zakłada, że koszt podróży z miasta A do miasta B jest dokładnie taki sam jak z miasta B do miasta A. Celem działania programu jest znalezienie najkrótszej (najtańszej) drogi w obu tych przypadkach.

W tym celu w programie zaimplementowano cztery algorytmy poszukujące rozwiązania: algorytm przeglądu zupełnego (*Brute-Force*), algorytm zachłanny Najbliższego Sąsiada (ang. *Nearest Neighbor* – NN), jego ulepszoną iteracyjną wersję, czyli Powtarzalnego Najbliższego Sąsiada (ang. *Repetitive Nearest Neighbor* – RNN), oraz algorytm losowy (ang. *Random Search* – RS). Ponadto aplikacja posiada funkcjonalność generowania losowych instancji problemu (macierzy) dla dowolnej liczby wierzchołków N .



Rysunek 1: Poglądowe przedstawienie problemu Komiwojażera

2 Opisy zaimplementowanych algorytmów

2.1 Przegląd zupełny (Brute-Force)

Cecha algorytmu	Wartość
Złożoność czasowa	$\mathcal{O}(n!)$

Tabela 1: Złożoność obliczeniowa algorytmu przeglądu zupełnego (Brute-Force)

Metoda przeglądu zupełnego polega na wygenerowaniu i sprawdzeniu wszystkich możliwych permutacji miast. W celu wydajnej generacji wykorzystano bibliotekę `std::next_permutation`, co pozwoliło na zablokowanie pierwszego wierzchołka i redukcję permutacji do $(n - 1)!$.

Zarys pseudokodu

```
min_koszt = 99999
Ustal miasto startowe na stale (np. miasto 0)
Dla kazdej mozliwej permutacji P pozostalych miast:
    koszt = ObliczKosztTrasy(P)
    Jezeli koszt < min_koszt:
        min_koszt = koszt
        najlepsza_trasa = P
Zwroc najlepsza_trasa oraz min_koszt
```

Zalety i wady

- **Zalety:** Gwarantuje odnalezienie absolutnego, globalnego optimum. Jego błąd zawsze wynosi 0%, dzięki czemu stanowi idealny punkt odniesienia (wzorzec) do testowania i oceny dokładności innych, szybszych algorytmów heurystycznych.
- **Wady:** Ponieważ ignoruje całkowicie logikę problemu i analizę kosztów poszczególnych krawędzi, dla większych instancji problemu szansa na wylosowanie trasy bliskiej optymalnej jest bliska zeru. Błąd rośnie logarytmicznie wraz z N .
- **Wady:** Złożoność silniowa $\mathcal{O}(n!)$ sprawia, że algorytm jest w praktyce bezużyteczny dla instancji problemu większych niż $N = 13$, z uwagi na eksplozję kombinatoryczną i ekstremalnie długi czas obliczeń.

2.2 Najbliższy Sąsiad (Nearest Neighbor - NN)

Cecha algorytmu	Wartość
Złożoność czasowa	$\mathcal{O}(n^2)$

Tabela 2: Złożoność obliczeniowa algorytmu Najbliższego Sąsiada (NN)

Algorytm zachłanny, który w każdym kroku wybiera krawędź o najmniejszej wadze prowadzącą do nieodwiedzonego jeszcze miasta.

Zarys pseudokodu

```
obecne_miasto = miasto_startowe
Oznacz obecne_miasto jako odwiedzone
Dopoki sa nieodwiedzone miasta:
    znajdz nieodwiedzone miasto (S) z najmniejszym kosztem
    przejścia
    koszt_calkowity += koszt przejścia do S
    obecne_miasto = S
    Oznacz S jako odwiedzone
koszt_calkowity += koszt powrotu do miasta startowego
Zwroc koszt_calkowity
```

Zalety i wady

- **Zalety:** Ekstremalnie szybki ($\mathcal{O}(n^2)$) i bardzo prosty w implementacji. Potrafi w ułamku sekundy wygenerować trasę dla tysięcy miast.
- **Wady:** Wykazuje duży błąd względem optimum. Podejmując lokalnie optymalne decyzje na początku, często zmuszony jest do wyboru bardzo kosztownych połączeń na samym końcu trasy (brak perspektywicznego planowania).

2.3 Powtarzalny Najbliższy Sąsiad (Repetitive Nearest Neighbor - RNN)

Cecha algorytmu	Wartość
Złożoność czasowa	$\mathcal{O}(n^3)$

Tabela 3: Złożoność obliczeniowa algorytmu Powtarzalnego Najbliższego Sąsiada (RNN)

Algorytm RNN jest rozszerzeniem metody NN. Uruchamia procedurę Najbliższego Sąsiada wielokrotnie, za każdym razem wybierając inne miasto jako punkt startowy.

Zarys pseudokodu

```
globalny_min_koszt = 99999
Dla kazdego miasta (i) od 0 do N-1:
    koszt_nn = UruchomNearestNeighbor(miasto_startowe = i)
    Jezeli koszt_nn < globalny_min_koszt:
        globalny_min_koszt = koszt_nn
Zwroc globalny_min_koszt
```

Zalety i wady

- **Zalety:** Stanowi świetny kompromis między czasem a dokładnością. Całkowicie niweluje problem "złego miasta startowego", drastycznie zmniejszając błąd względem zwykłego NN.
- **Wady:** Wymaga wykonania algorytmu zachłannego N razy, co podnosi złożoność do $\mathcal{O}(n^3)$. Wciąż nie gwarantuje znalezienia optimum globalnego.

2.4 Algorytm losowy (Random Search)

Cecha algorytmu	Wartość
Złożoność czasowa	$\mathcal{O}(k \cdot n)$

Tabela 4: Złożoność obliczeniowa algorytmu losowego (RS)

Algorytm generuje k (w programie ustalone na 1000) przypadkowych permutacji tras i wybiera spośród nich tę o najmniejszym koszcie. Złożoność jest zależna od ilości iteracji k oraz ilości miast n (zsumowanie trasy).

Zarys pseudokodu

```
min_koszt = 99999
Wykonuj k razy:
    P = WygenerujCalkowicieLosowaTrase()
    koszt = ObliczKosztTrasy(P)
    Jezeli koszt < min_koszt:
        min_koszt = koszt
Zwroc min_koszt
```

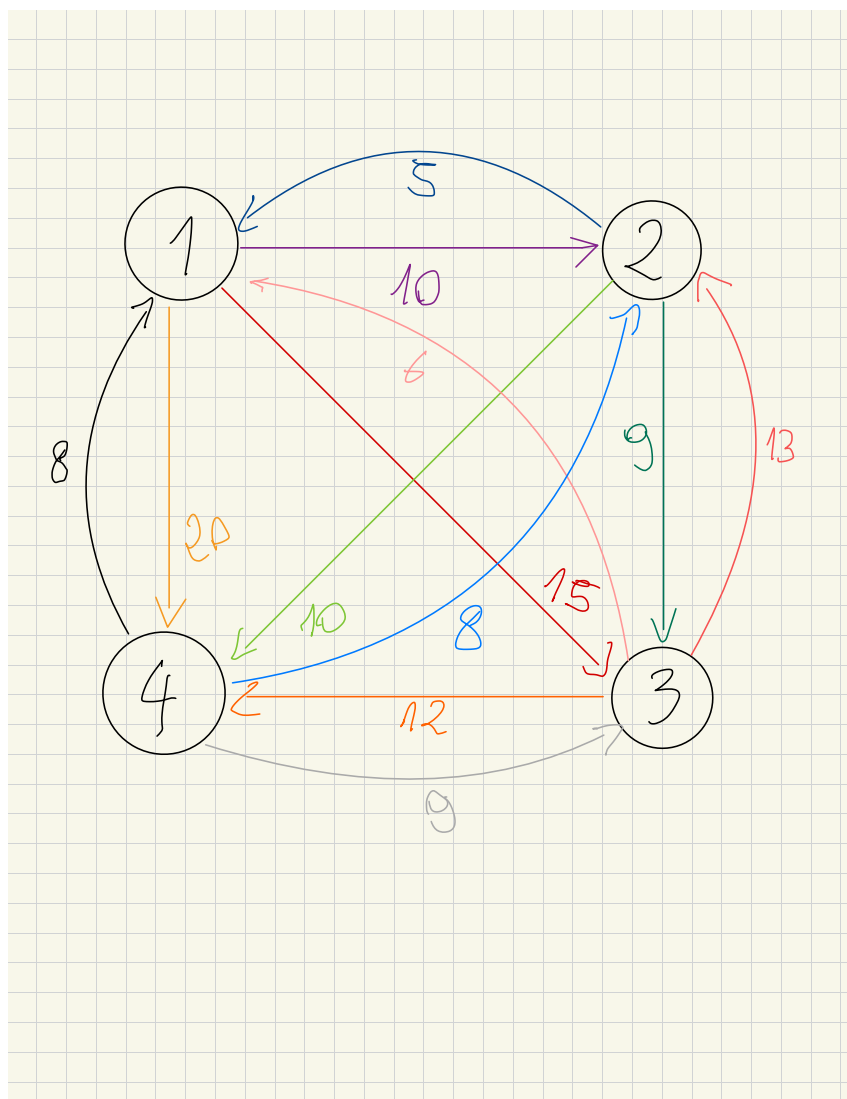
Zalety i wady

- **Zalety:** Odporny na wpadanie w lokalne minima (wynikające np. ze specyficznego ułożenia krawędzi w grafie). Zawsze znajduje "jakieś" rozwiązanie w z góry określonym czasie.
- **Wady:** Ponieważ ignoruje całkowicie logikę problemu (heurystyki, koszty poszczególnych krawędzi), dla większych instancji problemu szansa na wylosowanie trasy bliskiej optymalnej jest bliska zeru. Błąd rośnie logarytmicznie wraz z N .

3 Teoretyczny przykład działania algorytmów dla $N = 4$

Aby lepiej zobrazować mechanikę zaimplementowanych algorytmów dla problemu asymetrycznego (ATSP), przeanalizujemy prosty przykład dla 4 miast. Niech dana będzie następująca macierz kosztów przejść M (gdzie wiersze to wierzchołki startowe, a kolumny to wierzchołki docelowe):

$$M = \begin{bmatrix} \infty & 10 & 15 & 20 \\ 5 & \infty & 9 & 10 \\ 6 & 13 & \infty & 12 \\ 8 & 8 & 9 & \infty \end{bmatrix} \quad (1)$$



Rysunek 2: Ilustracyjne przedstawienie grafu skierowanego dla analizowanej macierzy $N = 4$

1. Działanie Przeglądu Zupełnego (Brute-Force)

Ustalając wierzchołek początkowy na 0 (dzięki optymalizacji redukującej liczbę permutacji), program musi sprawdzić dokładnie $(4 - 1)! = 3! = 6$ możliwych tras. Obliczamy całkowity koszt dla każdej z nich:

- $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0 : 10 + 9 + 12 + 8 = 39$
- $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0 : 10 + 10 + 9 + 6 = 35$ (**Rozwiązanie optymalne**)
- $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 0 : 15 + 13 + 10 + 8 = 46$
- $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0 : 15 + 12 + 8 + 5 = 40$
- $0 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 0 : 20 + 8 + 9 + 6 = 43$
- $0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0 : 20 + 9 + 13 + 5 = 47$

Algorytm Brute-Force bezbłędnie znajduje najkrótszą trasę o koszcie 35.

2. Działanie algorytmu Nearest Neighbor (NN)

Rozpatrzmy standardową metodę zachłanną startującą z wierzchołka 0:

1. Start w **0**. Szukamy najtańszego przejścia do nieodwiedzonego miasta. Koszty: do 1 (10), do 2 (15), do 3 (20). Wybieramy **1**.
2. Obecnie w **1**. Koszty do nieodwiedzonych: do 2 (9), do 3 (10). Wybieramy **2**.
3. Obecnie w **2**. Zostało tylko miasto **3** (koszt 12).
4. Obecnie w **3**. Wracamy do startu **0** (koszt 8).

Trasa wyznaczona przez NN: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$. Całkowity koszt: $10 + 9 + 12 + 8 = 39$. Widzimy tu klasyczny problem algorytmu zachłannego – podejmując lokalnie optymalne decyzje na początku (wybór 10 i 9), został on zmuszony do wybrania bardzo kosztownego powrotu na końcu trasy.

3. Działanie Repetitive Nearest Neighbor (RNN)

Aby rozwiązać problem "krótkowzroczności" metody NN, algorytm RNN uruchamia poszukiwania zachłanne dla każdego możliwego wierzchołka startowego:

- Start z 0: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ (koszt 39)
- Start z 1: $1 \rightarrow 0 \rightarrow 2 \rightarrow 3 \rightarrow 1$ (koszty: $5 + 15 + 12 + 8 = 40$)
- **Start z 2: $2 \rightarrow 0 \rightarrow 1 \rightarrow 3 \rightarrow 2$ (koszty: $6 + 10 + 10 + 9 = 35$)**
- Start z 3: $3 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ (koszty: $8 + 10 + 9 + 12 = 39$)

Dzięki iteracyjnemu sprawdzeniu każdego miasta startowego, RNN trafia na ścieżkę rozpoczynającą się od wierzchołka 2, co pozwala na skonstruowanie trasy o koszcie 35. Odpowiada ona permutacji $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$, czyli **rozwiązaniu optymalnemu**. Przykład ten idealnie obrazuje zasadność stosowania ulepszonej metody RNN.

4 Plan eksperymentu

W celu rzetelnej oceny zaimplementowanych algorytmów opracowano środowisko testowe, które automatyzuje proces badawczy. Eksperyment obejmuje analizę czasu wykonania oraz dokładności poszczególnych algorytmów przybliżonych.

4.1 Rozmiar używanych struktur danych

Do reprezentacji asymetrycznego problemu komiwojażera wykorzystano macierz kosztów w postaci dynamicznie alokowanej tablicy jednowymiarowej (`int* matrix = new int[N * N]`). Dostęp do elementów odbywa się za pomocą przesunięcia indeksu: $i \cdot N + j$. Taka liniowa reprezentacja w pamięci operacyjnej zapobiega fragmentacji i zapewnia optymalne wykorzystanie pamięci podręcznej procesora.

4.2 Sposób generowania danych testowych

Instancje problemu (macierze kosztów) generowane są w sposób zautomatyzowany. Koszty przejścia między poszczególnymi miastami losowane są z równomiernym prawdopodobieństwem z przedziału wartości całkowitych $\langle 1, 100 \rangle$. Aby uniemożliwić algorytmom pozostanie w tym samym mieście (lub powrót w jednym kroku), główna przekątna macierzy ($i = j$) wypełniana jest stałą wartością -1 , co w implementacji algorytmów traktowane jest jako nieskończoność (∞) i skutecznie odrzucane podczas szukania najmniejszej wagi.

4.3 Metoda pomiaru czasu i środowisko sprzętowe

Do precyzyjnego pomiaru czasu wykonywania poszczególnych algorytmów wykorzystano standardową bibliotekę C++ `<chrono>` oraz zegar `high_resolution_clock`. Czas mierzony jest bezpośrednio przed wywołaniem funkcji realizującej dany algorytm oraz natychmiast po jej zakończeniu, co wyklucza z pomiaru czas narzutu na generowanie macierzy. Wyniki podawane są w milisekundach (ms).

Pomiary zostały przeprowadzone na stacji roboczej, której szczegółową specyfikację sprzętową i programową przedstawiono w tabeli 5.

Komponent	Specyfikacja
Procesor (CPU)	AMD Ryzen 7 5700X3D 4.1 GHz
Karta graficzna (GPU)	NVIDIA GeForce RTX 3060 12 GB
Pamięć operacyjna (RAM)	32 GB DDR4 3200 mhz
Środowisko programistyczne	Visual Studio (C++) / Tryb Release

Tabela 5: Specyfikacja sprzętowa środowiska testowego

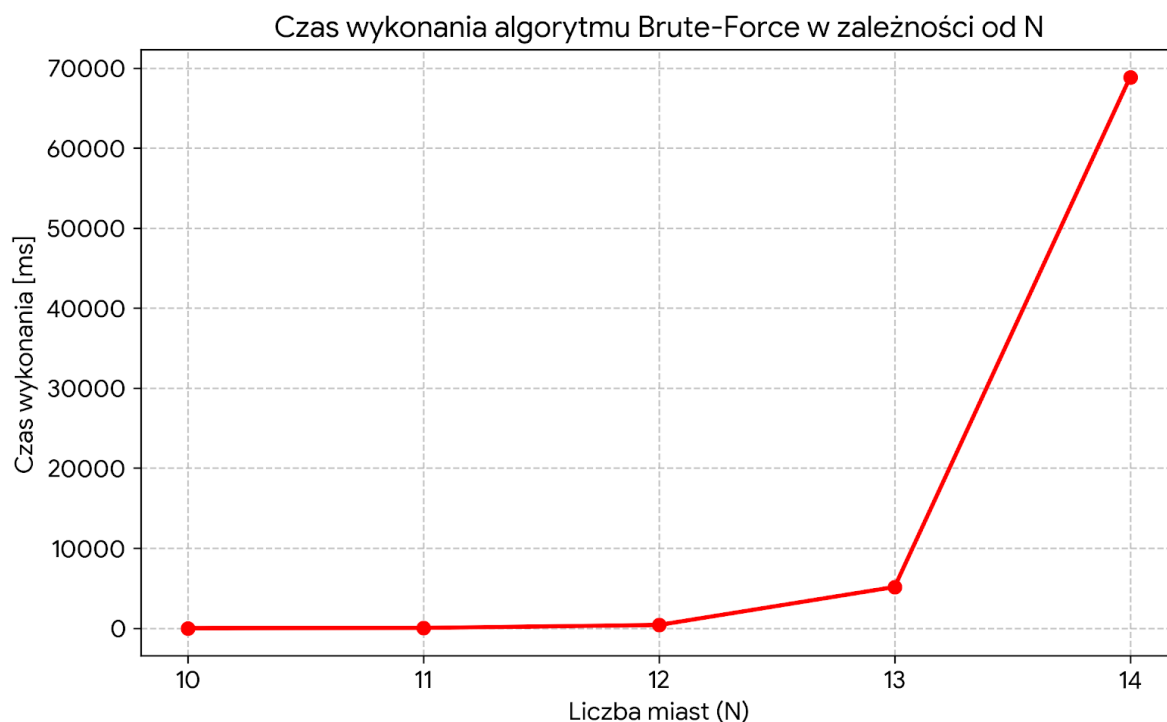
5 Wyniki eksperymentów

5.1 Zależność czasu wykonania od ilości miast dla algorytmu Brute-Force

Poniższa tabela oraz wykres (rys. Czas wykonania algorytmu Brute-Force w zależności od N) przedstawiają, jak gwałtownie rośnie czas znalezienia optymalnej trasy za pomocą algorytmu przeglądu zupełnego w miarę dodawania kolejnych wierzchołków do grafu.

Liczba miast (N)	Czas wykonania BF (ms)
10	3,16
11	33,79
12	400,74
13	5150,68
14	68863,80

Tabela 6: Czas wykonania algorytmu Brute-Force

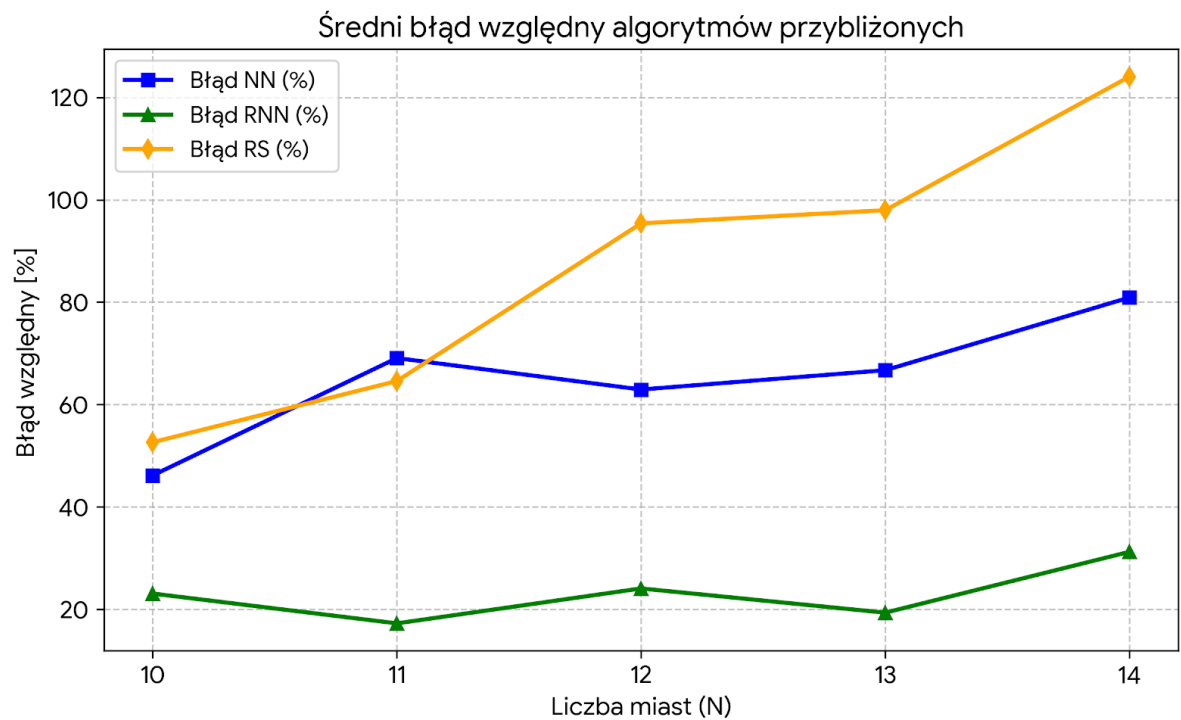


5.2 Błąd względny dla algorytmów NN, RNN oraz RS

Dla pozostałych algorytmów (NN, RNN oraz RS) zbadano wartość uśrednionego błędu względnego. Błąd ten obliczany był względem kosztu trasy optymalnej znalezionej przez algorytm Brute-Force, zgodnie ze wzorem: $\text{Błąd} = \frac{\text{KosztAlgorytmu} - \text{KosztOptymalny}}{\text{KosztOptymalny}} \cdot 100\%$. Wartości uśredniono na podstawie wielu niezależnych prób dla każdego N .

Liczba miast (N)	Błąd NN (%)	Błąd RNN (%)	Błąd RS (%)
10	46,17	23,10	52,65
11	69,08	17,28	64,60
12	62,94	24,10	95,39
13	66,71	19,37	97,95
14	80,91	31,25	124,03

Tabela 7: Uśredniony błąd względny algorytmów w zależności od rozmiaru instancji



6 Wnioski końcowe

Najlepszą z badanych metod przybliżonych okazał się algorytm **Powtarzalnego Najbliższego Sąsiada (RNN)**. **Przegląd zupełny (Brute-Force)**, chociaż jako jedyny zawsze pokaże dokładne i optymalne rozwiązanie, to ze względu na swoją ponadwielomianową, silniową złożoność obliczeniową absolutnie nie powinien być uznany za pierwszy wybór przy badaniu większych instancji problemu. Ciekawym przypadkiem w badaniu okazało się zestawienie podstawowej metody **Najbliższego Sąsiada (NN)** z jej rozszerzoną wersją (**RNN**). Chociaż dodanie iteracji po każdym możliwym wierzchołku startowym podnosi złożoność czasową o jeden stopień wielomianu, to na zestawieniu wyników jasno widać, że eliminacja problemu niefortunnego miasta początkowego drastycznie poprawia dokładność algorytmu i niweluje drastyczne błędy. Według literatury, jeśli do metody **RNN** dodano by dodatkowe mechanizmy lokalnej optymalizacji (jak np. algorytm *2-opt*), to w wielu przypadkach heurystyka ta generowałaby wyniki porównywalne z optimum globalnym, zachowując przy tym ułamek czasu wymaganego przez algorytmy dokładne. Z kolei **algorytm losowy (Random Search)** przy większych instancjach okazał się całkowicie nieefektywny, co potwierdza, że ślepe przeszukiwanie przestrzeni bez oparcia o wagi krawędzi mija się z celem.

Literatura

- [1] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C., *Wprowadzenie do algorytmów*, Wydawnictwo Naukowe PWN, Warszawa 2012.
- [2] Applegate D. L., Bixby R. E., Chvátal V., Cook W. J., *The Traveling Salesman Problem: A Computational Study*, Princeton University Press, 2006.
- [3] Wróblewski P., *Algorytmy, struktury danych i techniki programowania*, Wydawnictwo Helion, Gliwice 2015.
- [4] Papadimitriou C. H., *Złożoność obliczeniowa*, Wydawnictwa Naukowo-Techniczne, Warszawa 2002.
- [5] WilliamFiset, *Traveling Salesman Problem Dynamic Programming Directed Graph*, Adres URL: <https://www.youtube.com/watch?v=cY4HiiFH01o>
- [6] Nishu Jain, *Czym jest problem komiwojażera (TSP)? Przewodnik dla początkujących*, Adres URL: <https://zeorouteplanner.com/pl/przewodnik-po-problemach-komiwojazera/>